ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о проекте на тему:

Алгоритмы укладки паллет для реальных наборов данных

Выполнен студентами:

группы БПМИ 235, 3 курса Юхневич Егор Владимирович

Принял руководитель проекта:

Хельвас Александр Валериевич
Генеральный директор
ООО “СОККОС”, к.т.н.

Москва 2026

Реферат

ПАЛЛЕТИЗАЦИЯ, СКЛАДСКАЯ ЛОГИСТИКА, КОМБИНАТОРНАЯ ОПТИМИЗАЦИЯ, ТРЁХМЕРНАЯ УКЛАДКА, МЕТАЭВРИСТИКИ

Целью данной курсовой работы является разработка и программная реализация семейства алгоритмов трёхмерной укладки коробок на паллету, обеспечивающих максимальный коэффициент заполнения при контролируемой устойчивости размещения. Актуальность исследования определяется потребностью складской логистики в автоматизации процесса паллетизации: компактная и устойчивая укладка сокращает затраты на хранение, транспортировку и упаковочные материалы. В ходе работы разработаны пять алгоритмов: жадный алгоритм GreedySolver, эвристики Deepest Bottom Left Fill и First Fit Decreasing, метаэвристический алгоритм LNSSolver на основе поиска в большой окрестности с тринадцатью операторами мутации и генетический алгоритм GeneticSolver с популяционным подходом. Введены метрики качества: коэффициент заполнения, минимальный коэффициент опоры и баланс высот. Экспериментальное тестирование на 436 реальных задачах показало, что детерминированные эвристики достигают коэффициента заполнения 65–66% при времени менее 10 мс, а метаэвристические алгоритмы обеспечивают баланс между заполнением (63–67%) и устойчивостью (60–69% минимальной опоры). Реализована поддержка мультипаллетного режима с метрикой баланса высот.

Ключевые слова: паллетизация, трёхмерная укладка, метаэвристический алгоритм, поиск в большой окрестности, генетический алгоритм, коэффициент заполнения, устойчивость укладки.

Abstract

The purpose of this coursework is the development and software implementation of a family of algorithms for three-dimensional box packing on pallets, maximizing the fill rate while maintaining placement stability. Five algorithms were developed: a greedy algorithm (GreedySolver), Deepest Bottom Left Fill and First Fit Decreasing heuristics, a Large Neighborhood Search metaheuristic (LNSSolver) with thirteen mutation operators, and a population-based Genetic Algorithm (GeneticSolver). Quality metrics include fill rate (percolation), minimum support ratio, and height balance. Experimental evaluation on 436 real-world tasks demonstrated that deterministic heuristics achieve 65–66% fill rate in under 10 ms, while metaheuristic algorithms provide a balance between fill rate (63–67%) and stability (60–69% minimum support ratio). Multi-pallet mode with a height balance metric is supported.

Keywords: palletization, three-dimensional packing, metaheuristic algorithm, large neighborhood search, genetic algorithm, fill rate, packing stability.

Содержание

Введение 8
1 Постановка задачи 9
1.1 Подзадача 1. Укладка на одну паллету 9
1.2 Подзадача 2. Укладка на несколько паллет 10
1.3 Связь с классическими задачами комбинаторной оптимизации 10
2 Обзор литературы 11
2.1 Конструктивные эвристики 11
2.2 Метаэвристические подходы 12
2.3 Генетические алгоритмы 12
2.4 Поиск в большой окрестности (LNS) 12
2.5 Другие локальные методы 13
2.6 Точные методы и классификация задач 13
2.7 Методы машинного обучения 14
2.8 Смежные задачи 14
2.9 Патентный обзор 15
3 Формальная постановка задачи 16
3.1 Математическая модель 16
3.2 Формат входных данных 17
3.3 Формат выходных данных 17
3.4 Метрики качества укладки 17
3.5 Коэффициент заполнения (percolation) 18
3.6 Минимальный коэффициент опоры (min_support_ratio) 18
3.7 Баланс высот (height_balance) 19
3.8 Нормализованная координата центра масс 20
3.9 Целевая функция 20
3.10 Набор данных 20
4 Описание решения 22
4.1 Вспомогательная структура данных для хранения карты высот 22
4.2 Ограничение на устойчивость размещения 26
4.3 Жадный алгоритм укладки 26

4.4 Алгоритм DBLF (Deepest Bottom Left Fill) 27
4.5 Алгоритм FFD (First Fit Decreasing) 28
4.6 Метаэвристический алгоритм на основе поиска в большой окрестности 28
4.7 Генетический алгоритм 31
4.8 Укладка на несколько паллет 31
4.9 Метрики качества укладки 32
4.10 Результаты экспериментов 32
4.11 Сравнение LNSSolver и GeneticSolver при различных временных лимитах 37
4.12 Результаты в мультипаллетном режиме 39
4.13 Влияние веса баланса высот в мультипаллетном режиме 39
5 Анализ результатов 41
5.1 Сравнение алгоритмов для одной паллеты 41
5.2 Результаты в мультипаллетном режиме 41
Заключение 43
Список использованных источников 45

Глоссарий

Паллета — унифицированная платформа (EURO 1200 × 800 мм), на которой размещаются коробки для складирования и транспортировки.

Паллетизация — процесс укладки единичных грузовых мест (коробок) на паллету по определённому алгоритму.

SKU — (Stock Keeping Unit) — уникальный складской артикул товара, единица учёта на складе.

Коэффициент заполнения — (*percolation*) — отношение суммарного объёма размещённых коробок к объёму прямоугольного параллелепипеда, ограниченного паллетой и максимальной достигнутой высотой.

Минимальный коэффициент опоры — (*min_support_ratio*) — наименьшее среди всех коробок отношение площади основания, опирающейся на нижележащий слой, к полной площади основания.

Баланс высот — (*height_balance*) — метрика равномерности загрузки паллет при мультипаллетной укладке, основанная на коэффициенте вариации высот штабелей.

Карта высот — (HeightHandler) — вспомогательная структура данных, хранящая информацию о текущих высотах на поверхности паллеты.

Геном — (GenomHandler) — структура кодирования решения, определяющая порядок коробок, их ориентации, предпочтительные позиции и порог опоры.

Мутация — оператор изменения генома, порождающий новое решение из существующего; в работе реализовано тринадцать операторов.

JSON — текстовый формат для обмена и хранения данных.

API — программный интерфейс взаимодействия между компьютерными программами.

CSV — (*Comma-Separated Values*) — текстовый формат хранения табличных данных, используемый для входных и выходных файлов.

Обозначения и сокращения

ПО — программное обеспечение.

WMS — Warehouse Management System — складская информационная система.

LNS — Large Neighborhood Search — поиск в большой окрестности.

ALNS — Adaptive Large Neighborhood Search — адаптивный поиск в большой окрестности.

GA — Genetic Algorithm — генетический алгоритм.

DBLF — Deepest Bottom Left Fill — эвристика заполнения от глубокого нижнего левого угла.

FFD — First Fit Decreasing — алгоритм “Первый подходящий с убыванием”, при котором объекты сортируются от большего к меньшему.

BL — Bottom-Left — эвристика “Нижний-Левый”, стратегия размещения в самую нижнюю и самую левую доступную позицию.

3D-BPP — Three-Dimensional Bin Packing Problem — задача трёхмерной упаковки в контейнеры.

RL — Reinforcement Learning — обучение с подкреплением.

MIP — Mixed-Integer Programming — смешанное целочисленное программирование.

CP — Constraint Programming — программирование ограничений.

CoM — Center of Mass — центр масс.

SKU — Stock Keeping Unit — уникальный складской артикул товара.

CSV — Comma-Separated Values — формат хранения табличных данных.

Введение

Паллетизация остаётся ключевым технологическим узлом современной складской логистики: именно от плотности и устойчивости укладки зависит стоимость хранения, расход упаковочных материалов и объём перевозимого груза. С ростом ассортимента e-commerce-складов задача становится всё более вариативной, а ручная разработка правил паллетизации — трудоёмкой и часто приводит к недогрузам до 50 %.

Цель работы — разработка и программная реализация семейства алгоритмов трёхмерной укладки коробок на паллету, обеспечивающих максимальный коэффициент заполнения при производительном времени расчёта.

В первом разделе работы содержится постановка задачи и подзадач, на которые она была разбита.

Во втором разделе работы приведен обзор научно - технической литературы по теме исследований.

Третий раздел формализует задачу и описывает исходные данные, используемые для построения алгоритмов.

В четвертом разделе приведены описания разработанных алгоритмов: жадного алгоритма GreedySolver, эвристик DBLF и FFD, метаэвристического алгоритма LNSSolver на основе поиска в большой окрестности и генетического алгоритма GeneticSolver. Описан мультипаллетный режим с метрикой баланса высот.

В пятом разделе приведён сравнительный анализ разработанных алгоритмов на 436 реальных задачах в однопаллетном и мультипаллетном режимах.

1 Постановка задачи

Задача автоматической паллетизации возникает в контексте складской логистики, где требуется разместить набор разнородных коробок на стандартизированном поддоне (паллете) ограниченных размеров. Качество полученной укладки определяется двумя конкурирующими критериями: плотностью заполнения объёма и физической устойчивостью штабеля. В данной работе решаются две связанные подзадачи.

1.1 Подзадача 1. Укладка на одну паллету

Дан набор коробок $S = \{b_1, b_2, \ldots, b_K\}$, каждая из которых характеризуется целочисленными размерами (l_i, w_i, h_i) и массой m_i. Дана паллета с прямоугольным основанием $L \times W$ и нормировочной высотой H.

Требуется определить для каждой коробки координаты размещения (x_i, y_i, z_i) и ориентацию таким образом, чтобы:

а) все коробки помещались внутри области $[0, L] \times [0, W]$ по основанию;

б) коробки не пересекались попарно;

в) каждая коробка опиралась либо на поверхность паллеты ($z_i = 0$), либо на верхнюю грань хотя бы одной ранее уложенной коробки;

г) коэффициент заполнения (percolation) был максимален;

д) укладка удовлетворяла ограничению на устойчивость.

Допустимыми являются только повороты вокруг вертикальной оси (перестановка l_i и w_i), поскольку переворачивать коробки в практических сценариях недопустимо.

Ограничение устойчивости формализуется через минимальный коэффициент опоры (min_support_ratio). Для каждой коробки вычисляется доля площади её нижней грани, которая опирается на нижележащий слой. Минимальное значение этой доли по всем коробкам определяет устойчивость укладки в целом. Подробное обоснование выбора данной метрики приведено в разделе 3.6.

1.2 Подзадача 2. Укладка на несколько паллет

В практической ситуации для одного адреса доставки может потребоваться несколько паллет. Дан тот же набор коробок S и количество доступных паллет $P \geqslant 2$, каждая с одинаковым основанием $L \times W$.

Помимо требований, перечисленных для подзадачи 1, предъявляется дополнительное условие: высоты штабелей на разных паллетах должны быть по возможности одинаковыми. Неравномерная загрузка приводит к неэффективному использованию кузова транспортного средства и повышенному риску повреждения при перевозке.

Равномерность загрузки оценивается метрикой баланса высот (height_balance), которая принимает значение 1 при одинаковых высотах и убывает при увеличении разброса (формальное определение — в разделе 3.7).

1.3 Связь с классическими задачами комбинаторной оптимизации

Задача паллетизации является частным случаем задачи трёхмерной упаковки (3D Bin Packing Problem, 3D-BPP), которая принадлежит классу NP-трудных задач [1]. Типология Wäscher et al. [2] относит данную постановку к классу *Single Bin-Size Bin Packing Problem* при нескольких паллетах и *Single Large Object Placement Problem* при одной паллете.

Существенным отличием от абстрактной 3D-BPP является наличие дополнительных практических ограничений: требование устойчивости, ограниченность поворотов, неоднородность коробок по весу и категории товара. Обзор таких ограничений систематизирован в работе [3].

2 Обзор литературы

Трёхмерная задача упаковки изучается на протяжении нескольких десятилетий. За это время накоплен обширный арсенал методов, который целесообразно разделить на четыре направления: конструктивные эвристики, метаэвристические подходы, точные и гибридные методы, а также методы на основе машинного обучения.

2.1 Конструктивные эвристики

Конструктивные эвристики формируют решение за один проход, последовательно размещая объекты по определённому правилу. Их главное достоинство — скорость, а основной недостаток — невозможность пересмотра ранее принятых решений.

Karabulut и İnceoğlu [4] предложили метод Deepest Bottom Left Fill (DBLF), при котором каждая очередная коробка помещается в самую глубокую (низкую) позицию, а при равных высотах — как можно левее. DBLF используется ими как декодирующая процедура в составе гибридного генетического алгоритма; авторы сравнили четыре функции приспособленности и установили, что критерий минимизации максимальной высоты контейнера даёт наилучшую плотность заполнения.

Wei et al. [5] описали эвристику Skyline для двумерной задачи strip-packing: свободное пространство моделируется профилем «линии неба», а объект размещается в первую подходящую впадину. Метод обеспечивает сложность $O(n \log n)$ и впоследствии был обобщён на трёхмерный случай.

Parreño et al. [6] разработали алгоритм Maximal-Space, в котором свободный объём контейнера представляется множеством перекрывающихся «максимальных пространств», обновляемых после каждого размещения. На тестах Bischoff–Ratcliff метод улучшил лучшие известные решения для 25 из 30 инстансов.

Классическая стратегия First Fit Decreasing (FFD) [7], при которой объекты предварительно сортируются по убыванию размера, широко применяется как базовый строительный блок более сложных алго-

ритмов. Практическое руководство Jylänki [8] систематизирует свыше двадцати двумерных эвристик размещения и предоставляет открытую C++-библиотеку RectangleBinPack.

2.2 Метаэвристические подходы

Метаэвристики итеративно улучшают одно или несколько решений, не гарантируя оптимальность, но обеспечивая высокое качество за ограниченное время.

2.3 Генетические алгоритмы

Тимофеева, Чернышева, Волков и Корелин [9] адаптировали генетический алгоритм для трёхмерной упаковки блоков в контейнер. Авторы применили турнирную и рулеточную селекцию, модифицированный кроссинговер и мутацию перемещением гена в начало приоритетного списка.

Zhang et al. [10] усилили генетический алгоритм генеративной состязательной сетью (GAN), которая создаёт перспективные хромосомы, ускоряя сходимость. Подход сократил неиспользованный объём на 2–3 п.п. относительно стандартного GA.

В [11, 12] Гиля-Зетинов, Хельвас и Панкратов исследовали эвристики слоёв и генетический алгоритм для задачи паллетизации. Ими были предложены критерии качества укладки на основе коэффициента перколяции и коэффициента устойчивости, а также показана слабая чувствительность качества к точности задания размеров коробок.

2.4 Поиск в большой окрестности (LNS)

Pisinger и Ropke [13] предложили Large Neighborhood Search (LNS) — метаэвристику, которая на каждой итерации разрушает часть текущего решения и восстанавливает его заново. На задачах маршрутизации и упаковки LNS продемонстрировал существенное превосходство над классическими локальными поисками благодаря способности покидать локальные оптимумы.

Ropke и Pisinger [14] развили идею адаптивного варианта ALNS, в котором веса операторов разрушения и восстановления подстраиваются в ходе поиска. На задачах pickup-and-delivery ALNS улучшил лучшие известные решения для 40% инстансов.

Shaw [15] впервые сформулировал constraint-based подход к LNS: разрушаемые переменные выбираются на основе их связанности (relatedness) с «неудачными» частями решения. Эта идея легла в основу множества современных операторов разрушения.

Обзор Pisinger и Ropke [16] систематизирует архитектуру LNS, выделяя выбор размера окрестности, стратегии принятия решений (simulated annealing, threshold accepting, record-to-record) и гибридизацию с точными методами.

Ekici [17] применил LNS к задаче bin packing с конфликтами между предметами, показав, что адаптивные операторы разрушения обеспечивают преимущество на инстансах средней и большой размерности.

2.5 Другие локальные методы

Faroe, Pisinger и Zachariasen [18] использовали Guided Local Search, при котором штрафы за перегруженные измерения постепенно снижаются, позволяя приближаться к нижней границе.

Calzavara et al. [19] построили MIP-модель паллетизации с практическими ограничениями (выступы, устойчивость) и разработали гибрид GRASP с локальным поиском. На реальных данных высота паллет была снижена на 6% относительно промышленных правил укладки.

2.6 Точные методы и классификация задач

Martello, Pisinger и Vigo [1] предложили точный алгоритм на основе branch-and-bound в сочетании с эвристикой L2, достигающий оптимума для задач с числом коробок до 100. Предложенные ими инстансы остаются стандартным бенчмарком.

Queiroz et al. [20] разработали динамические и колонковые алгоритмы для трёхмерных guillotine-задач (knapsack, cutting-stock, strip-packing).

Amossen и Pisinger [21] ввели guillotine-ограничения в многомерное упаковывание и предложили CP-модель, показав рост потерь объёма лишь на 0,5% при полной технологичности разрезов.

Типология Wäscher, Haußner и Schumann [2] расширила классификацию Dyckhoff, выделив четыре уровня описания задач cutting & packing; данная типология стала стандартом при формулировке новых постановок.

Обзор Bortfeldt и Wäscher [3] систематизирует ограничения контейнерной загрузки и охватывает период 1995–2013 годов. Bischoff и Ratcliff [22] анализируют влияние практических ограничений (несовместимость грузов, очаговые нагрузки) на выбор моделей и эвристик.

2.7 Методы машинного обучения

Fang, Rao и Shi [23] применили глубокое обучение с подкреплением (Actor–Critic) для двумерного strip-packing. Сеть обучается без учителя и оптимизирует длину ленты, превосходя классические эвристики на 3 п.п. Подход демонстрирует перспективность RL-методов для задач размещения.

Zhao et al. [24] разработали RL-политику для online-сценария трёхмерной упаковки с гарантией устойчивости на основе stacking-tree, получив выигрыш до 15% по числу контейнеров относительно rule-based baseline.

Ali et al. [25] в обзорной работе систематизировали offline- и online-алгоритмы трёхмерной упаковки, классифицировав ограничения и метрики задержки.

2.8 Смежные задачи

Ananno и Ribeiro [26] решали многоконтейнерную 3D-BPP с реальными ограничениями, применив двухэтапный подход (зонная деком-

позиция и шаблонный поиск), сократив число контейнеров в среднем на один.

Alonso et al. [27] объединили задачу паллетизации с задачей погрузки автомобилей в междеповской логистике, получив экономию транспортных затрат 4–7%.

Yang et al. [28] решали задачу оптимизации размеров упаковки для электронной коммерции; их MIP-модель позволила сократить расход картона на 8% и «воздушный» объём — на 12%.

McIlroy [29] исследовал алгоритм оптимистической сортировки, эффективный для частично упорядоченных данных; подобная предварительная сортировка часто применяется в задачах упаковки при построении начальных решений.

2.9 Патентный обзор

Baumann [30] запатентовал метод автоматического формирования сборной паллеты из коробок, находящихся в нескольких однородных паллетах. Pankratov, Ehrenberg и Sweet [31] описали систему послойного формирования паллеты с программным управлением.

3 Формальная постановка задачи

В данной главе вводятся математические обозначения, описываются форматы входных и выходных данных, а также формализуются метрики качества укладки.

3.1 Математическая модель

Паллета моделируется прямоугольным параллелепипедом с основанием $L \times W$ и нормировочной высотой H. В экспериментах используется стандартная паллета EUR 1200×800 мм с нормировочной высотой $H = 2200$ мм.

Дан набор из K типов коробок. Коробка типа i характеризуется размерами (l_i, w_i, h_i), массой m_i и количеством экземпляров q_i. Общее число коробок составляет $N = \sum_{i=1}^{K} q_i$.

Решением задачи является отображение каждого экземпляра коробки в позицию и ориентацию на паллете. Позиция j-го экземпляра задаётся координатами двух противоположных вершин: нижней левой (x_j, y_j, z_j) и верхней правой (X_j, Y_j, Z_j), где $X_j - x_j$ и $Y_j - y_j$ — длина и ширина коробки в выбранной ориентации, а $Z_j - z_j = h_i$ — высота.

Допускается единственный тип вращения — поворот на 90 вокруг вертикальной оси, при котором l_i и w_i меняются местами. Переворот коробки (перестановка высоты с длиной или шириной) запрещён.

Решение является допустимым, если выполнены следующие условия:

а) **Вложенность:** $0 \leqslant x_j$, $X_j \leqslant L$, $0 \leqslant y_j$, $Y_j \leqslant W$ для всех j;

б) **Непересечение:** для любых двух коробок $j \neq k$ их объёмные проекции не пересекаются;

в) **Опора:** $z_j = 0$ (коробка стоит на паллете) либо z_j совпадает с верхней гранью Z_k хотя бы одной коробки k, проекция которой перекрывает точку $(x_j + l/2,\, y_j + w/2)$.

3.2 Формат входных данных

Входные данные представляют собой CSV-файл, каждая строка которого описывает один тип коробок:

`SKU, Quantity, Length, Width, Height, Weight, Strength, Aisle, Caustic`

где:

— **SKU** — уникальный идентификатор товарной позиции;
— **Quantity** — количество экземпляров данного типа;
— **Length**, **Width**, **Height** — размеры коробки в миллиметрах;
— **Weight** — масса коробки в граммах;
— **Strength** — прочность (в текущей версии не используется);
— **Aisle** — номер секции магазина (категория товара);
— **Caustic** — признак едкого содержимого (в текущей версии не используется).

3.3 Формат выходных данных

Результат укладки описывается CSV-файлом, каждая строка которого задаёт позицию одной коробки:

`Pallet, SKU, x, y, z, X, Y, Z`

где `Pallet` — номер паллеты (от 0), `SKU` — идентификатор типа коробки, (x, y, z) — координаты нижнего левого угла, (X, Y, Z) — координаты верхнего правого угла. Порядок строк определяет последовательность укладки.

3.4 Метрики качества укладки

Для оценки решения используется набор метрик, каждая из которых отражает один из аспектов качества: плотность заполнения, устойчивость и равномерность загрузки.

3.5 Коэффициент заполнения (percolation)

Коэффициент заполнения определяется как отношение суммарного объёма уложенных коробок к объёму паллеты:

$$\text{percolation} = \frac{\sum_{j=1}^{N} V_j}{L \times W \times H_{\max}},$$

где $V_j = (X_j - x_j)(Y_j - y_j)(Z_j - z_j)$ — объём j-й коробки, а $H_{\max} = \max_j Z_j$ — фактическая высота штабеля. В случае нескольких паллет перколяция вычисляется как совокупная по всем паллетам: сумма объёмов коробок делится на сумму объёмов паллет.

Данная метрика принимает значения в диапазоне $(0, 1]$. Чем выше перколяция, тем эффективнее используется объём паллеты.

3.6 Минимальный коэффициент опоры (min_support_ratio)

Устойчивость укладки оценивается через минимальный коэффициент опоры — наименьшую среди всех коробок долю площади нижней грани, опирающуюся на нижележащий слой.

Для коробки j с нижней гранью площадью $A_j = (X_j - x_j)(Y_j - y_j)$ коэффициент опоры определяется как:

$$\text{support_ratio}_j = \frac{A_j^{\text{supported}}}{A_j},$$

где $A_j^{\text{supported}}$ — площадь области нижней грани коробки j, которая опирается на верхнюю грань нижележащих коробок (для коробок на поверхности паллеты $\text{support_ratio}_j = 1$). Метрика всей укладки:

$$\text{min_support_ratio} = \min_{j=1}^{N} \text{support_ratio}_j.$$

Обоснование выбора метрики. В промышленной практике для оценки устойчивости используется ряд подходов: ограничение на до-

лю площади опоры [22], проверка нахождения центра тяжести внутри полигона опоры [19], stacking-tree с физическим моделированием [24].

Метрика min_support_ratio выбрана по нескольким причинам:

— **Вычислительная эффективность.** Расчёт площади опоры реализуется через структуру HeightHandler за время $O(n)$ для каждой коробки, что не увеличивает асимптотику основного алгоритма.

— **Дифференцируемость по решению.** Метрика непрерывно зависит от расположения коробок, что позволяет использовать её в составе целевой функции метаэвристических алгоритмов с градиентоподобной оптимизацией.

— **Интерпретируемость.** Значение 0,25 означает, что хотя бы одна коробка опирается лишь четвертью основания, что легко визуализировать и оценить.

— **Чувствительность к наихудшему случаю.** Операция минимума гарантирует, что ни одна коробка не «повисает» — в отличие от усреднённых метрик, которые допускают наличие полностью неопёртых коробок при высоком среднем значении.

Дополнительно в алгоритме применяется ограничение на поддержку центра масс (см. раздел 4): коробка может быть уложена только в позицию, где геометрический центр её основания проецируется на опорную поверхность. Это ограничение пассивно поднимает min_support_ratio до значений не менее 0,25.

3.7 Баланс высот (height_balance)

При укладке на несколько паллет важна равномерность загрузки. Метрика баланса высот определяется через коэффициент вариации высот штабелей:

$$\text{height_balance} = \max\left(0,\, 1 - \frac{\sigma(H_1, \ldots, H_P)}{\mu(H_1, \ldots, H_P)}\right),$$

где H_i — высота штабеля на i-й паллете, μ — среднее, σ — стандартное отклонение. При $P = 1$ метрика равна 1.

Значение 1 соответствует идеально равномерной загрузке, а значения ниже 0,8 указывают на существенный дисбаланс, при котором одна паллета нагружена значительно больше других.

3.8 Нормализованная координата центра масс

Для оценки распределения веса по высоте используется нормализованная z-координата центра масс:

$$\mathrm{CoM}_z^{\mathrm{rel}} = \frac{\sum_{j=1}^{N} m_j \cdot \frac{z_j + Z_j}{2}}{\left(\sum_{j=1}^{N} m_j\right) \cdot H_{\max}}.$$

Низкие значения $\mathrm{CoM}_z^{\mathrm{rel}}$ (ближе к 0) соответствуют укладке, в которой тяжёлые коробки расположены внизу, что повышает физическую устойчивость штабеля при транспортировке. Метрика входит в целевую функцию с отрицательным знаком: оптимизируется $(1 - \mathrm{CoM}_z^{\mathrm{rel}})$.

3.9 Целевая функция

Все метрики объединяются в скалярную целевую функцию:

$$\mathrm{score} = w_1 \cdot \mathrm{percolation} + w_2 \cdot \mathrm{min_support_ratio} + w_3 \cdot (1 - \mathrm{CoM}_z^{\mathrm{rel}}) + w_4 \cdot \mathrm{height_balance},$$

где $w_1, \ldots, w_4$ — настраиваемые неотрицательные веса. Для задачи с одной паллетой типичными являются значения $w_1 = 1$, $w_2 = 2$, $w_3 = w_4 = 0$, подобранные экспериментально (см. раздел 4).

3.10 Набор данных

Экспериментальное тестирование проводилось на наборе из 436 реальных задач, предоставленных компанией-оператором складской логистики. Каждая задача содержит от 3 до 120 типов коробок (от 5 до 400 экземпляров) и охватывает широкий диапазон размеров: от $50 \times 50 \times 30$ мм до $600 \times 400 \times 400$ мм. Масса отдельных коробок варьируется от 100 г до 25 кг.

Для мультипаллетного режима из одиночных задач формируются составные тесты путём случайного объединения нескольких заказов в одну группу доставки. Количество паллет в составных тестах варьируется от 2 до 6.

4 Описание решения

В данном разделе описывается разработанный алгоритм укладки коробок на паллету, включающий вспомогательные структуры данных и метаэвристические методы оптимизации.

4.1 Вспомогательная структура данных для хранения карты высот

Введём вспомогательную структуру данных HeightHandler, предназначенную для хранения информации о текущих высотах на поверхности паллеты. Данная структура является ключевым компонентом алгоритма, поскольку операции с ней выполняются при размещении каждой коробки.

Структура HeightHandler поддерживает четыре основные операции. Операция `get_h`(x_1, y_1, x_2, y_2) возвращает максимальную высоту в прямоугольной области $[x_1, x_2] \times [y_1, y_2]$. Операция `get_area`$(x_1, y_1, x_2, y_2)$ возвращает площадь, находящуюся на максимальной высоте в заданном прямоугольнике; данная операция используется для расчёта площади опоры коробки. Операция `add_rect`(x_1, y_1, x_2, y_2, h) добавляет новый прямоугольник с заданной высотой h, обновляя карту высот. Операция `get_dots`(box) возвращает множество позиций, являющихся потенциальными точками размещения для коробки заданного размера.

Операция `get_dots` формирует множество потенциальных позиций на основе углов уже размещённых прямоугольников. Такой подход позволяет сократить пространство поиска с $O(W \times L)$ до $O(n)$, где W и L — размеры паллеты, а n — количество уже уложенных коробок.

В рамках работы были исследованы пять различных реализаций структуры HeightHandler, различающихся алгоритмическими подходами: наивная реализация Baseline, оптимизированный список Rects, плотная двумерная сетка Grid, двумерное дерево отрезков SegTree и квадродерево Quadtree.

Реализация Baseline представляет собой простейший вариант, в котором все добавленные прямоугольники хранятся в массиве. При выполнении запросов `get_h` и `get_area` производится полный перебор всех элементов массива. Вычислительная сложность операций `get_h` и `get_area` составляет $O(n)$, тогда как операция `add_rect` выполняется за константное время $O(1)$. Корректность данной реализации гарантируется свойством задачи паллетизации: прямоугольники добавляются послойно, поэтому новый прямоугольник всегда имеет высоту строго большую, чем существующие прямоугольники в той же области.

Реализация Rects является оптимизированной версией Baseline, в которой прямоугольники поддерживаются отсортированными по убыванию высоты. Такая организация данных позволяет завершить поиск максимума при первом обнаружении пересечения с запрашиваемой областью. Вычислительная сложность операций `get_h` и `get_area` в худшем случае остаётся $O(n)$, однако на практике наблюдается существенное ускорение за счёт раннего выхода из цикла. Сложность операции `add_rect` составляет $O(n)$ вследствие необходимости поддержания упорядоченности.

Реализация Grid разбивает паллету на ячейки фиксированного размера 10×10 мм. В каждой ячейке хранится максимальная высота. Для стандартной паллеты размером 1200×800 мм структура содержит $120 \times 80 = 9600$ ячеек. Вычислительная сложность всех операций составляет $O(k)$, где k — количество ячеек, затрагиваемых запросом. Недостатком данной реализации является значительный объём используемой памяти и медленное обновление при добавлении крупных прямоугольников.

Реализация SegTree основана на двумерном дереве отрезков с отложенным обновлением (Lazy Propagation). Структура организована как X-дерево, каждый узел которого содержит Y-дерево. Вычислительная сложность операций `get_h` и `get_area` составляет $O(\log W \cdot \log H)$, однако операция `add_rect` требует $O(\log W \cdot H)$ времени вследствие необходимости объединения Y-деревьев. Несмотря на асимптотическую эф-

фективность для запросов, высокая константа и сложность операции обновления снижают практическую производительность.

Реализация Quadtree использует рекурсивное разбиение пространства на четыре части. Каждый узел хранит максимальную высоту в своей области. При частичном пересечении с областью обновления узел разбивается на четыре дочерних. Вычислительная сложность операций `get_h` и `add_rect` составляет $O(\log(\max(W,H)))$ при равномерном распределении, однако операция `get_area` требует $O(n)$ времени. Недостатком являются высокие накладные расходы на управление памятью и частое разбиение узлов.

На рисунке 4.1 представлена визуализация карты высот, хранимой структурой HeightHandler. Карта высот отображает вид сверху паллеты, где цвет прямоугольников кодирует высоту: синие области соответствуют низким уровням, зелёные — высоким. Данная визуализация демонстрирует, как HeightHandler отслеживает текущее состояние поверхности паллеты.

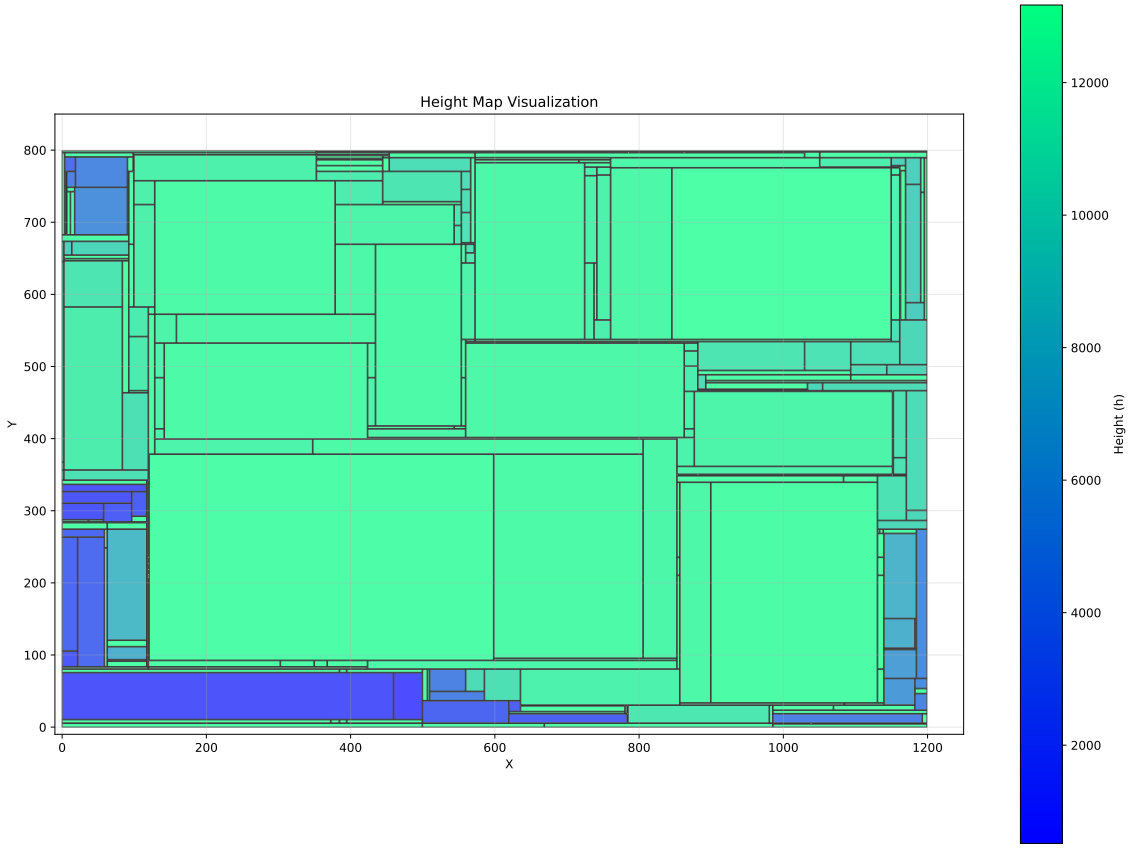

Рисунок 4.1 — Карта высот HeightHandler: вид сверху с цветовым кодированием высоты (синий — низкие области, зелёный — высокие)

Сравнительный анализ вычислительной сложности реализаций приведён в таблице 4.1.

Таблица 4.1 — Сравнение вычислительной сложности реализаций HeightHandler

Реализация	`get_h`	`get_area`	`add_rect`
Baseline	$O(n)$	$O(n)$	$O(1)$
Rects	$O(n)$	$O(n)$	$O(n)$
Grid	$O(k)$	$O(k)$	$O(k)$
SegTree	$O(\log W \log H)$	$O(\log W \log H)$	$O(\log W \cdot H)$
Quadtree	$O(\log(\max(W,H)))$	$O(n)$	$O(\log(\max(W,H)))$

Экспериментальное тестирование проводилось на 436 реальных задачах с суммарным количеством операций 27,5 миллионов. Результаты представлены в таблице 4.2. Реализация Rects продемонстрировала наилучшую производительность. Данный результат объясняется эффективным использованием раннего выхода при сортировке по убыванию высоты, низкими накладными расходами простой структуры данных и эффективным использованием кэш-памяти процессора.

Таблица 4.2 — Результаты тестирования реализаций HeightHandler

Реализация	**Всего, мс**	`get_h`, мс	`get_area`, мс	`add_rect`, мс
Baseline	36412	19724	21231	4
Rects	1658	1713	2121	14
Grid	120688	32015	93547	112
SegTree	71199	34877	33292	5825
Quadtree	168971	86822	81857	885

Реализации Grid, SegTree и Quadtree оказались медленнее базовой реализации. Причинами являются высокие накладные расходы на управление структурами данных, а также тот факт, что типичные коробки покрывают значительную часть паллеты, что нивелирует преимущества иерархических структур. В финальной версии алгоритма используется реализация Rects.

4.2 Ограничение на устойчивость размещения

Для обеспечения устойчивости укладки введено ограничение: коробка может быть размещена в позиции (x, y) только в том случае, если центр масс основания коробки опирается на нижележащую коробку или на поверхность паллеты. Формально, если точка $(x + \frac{length}{2}, y + \frac{width}{2})$ находится на высоте h, то высота поверхности в этой точке также должна быть равна h.

Данное ограничение пассивно увеличивает минимальный коэффициент опоры (min_support_ratio) до значений порядка 0.25–0.30, поскольку коробки не могут «висеть» над пустотой. Без этого ограничения минимальный коэффициент опоры может падать до 0.01–0.03.

4.3 Жадный алгоритм укладки

Разработанный жадный алгоритм GreedySolver реализует последовательный подход к укладке коробок. Алгоритм обрабатывает коробки в порядке их появления во входных данных, выбирая для каждой коробки позицию с минимальной итоговой высотой.

На каждом шаге алгоритм получает очередную коробку из входного списка и рассматривает допустимые ориентации данной коробки. Поскольку коробки могут вращаться только вокруг вертикальной оси, допускается перестановка длины и ширины основания (length и width), что даёт две возможные ориентации для несимметричных коробок. Для каждой ориентации структура HeightHandler возвращает множество потенциальных позиций размещения. Позиции, не удовлетворяющие ограничению на устойчивость, отфильтровываются. Для оставшихся позиций вычисляется оценка качества как сумма текущей высоты поверхности и высоты размещаемой коробки. Выбирается позиция с минимальной оценкой, после чего коробка размещается и структура HeightHandler обновляется.

Псевдокод жадного алгоритма представлен в алгоритме 1. Вычислительная сложность алгоритма составляет $O(n \cdot k \cdot m)$, где n — коли-

чество коробок, k — количество рассматриваемых ориентаций (не более 2), m — количество потенциальных позиций размещения.

Algorithm 1: Greedy packing algorithm

```
Input: boxes – list of boxes, pallet – pallet dimensions
Output: answer – placement positions for all boxes
heightHandler ← create height map structure;
foreach box ∈ boxes do
    bestScore ← ∞;
    foreach rotatedBox ∈ getRotations(box) do
        foreach (x, y) ∈ heightHandler.getDots(rotatedBox)
         do
            if not isCenterOfMassSupported(x, y, rotatedBox)
             then
                continue;
            h ← heightHandler.getH(x, y, x +
             rotatedBox.length, y + rotatedBox.width);
            score ← h + rotatedBox.height;
            if score < bestScore then
                save best position (x, y, h) and rotation;
    add box to answer at best position;
    heightHandler.addRect(bestPosition, box.height);
return answer
```

4.4 Алгоритм DBLF (Deepest Bottom Left Fill)

Алгоритм DBLFSolver реализует стратегию Deepest Bottom Left Fill [4], отличающуюся от жадного алгоритма критерием выбора позиции. Вместо минимизации итоговой высоты $h_0 + h_{box}$ алгоритм DBLF минимизирует лексикографически кортеж $(h_0,\ x,\ y,\ h_0 + h_{box})$, где h_0 — высота основания в точке размещения. Такой критерий обеспечивает размещение коробок как можно глубже (ниже), а при равной глубине — как можно ближе к нижнему левому углу паллеты.

В отличие от GreedySolver, алгоритм DBLF не проверяет ограничение на устойчивость (поддержку центра масс), что позволяет достигать более высокого коэффициента заполнения, но приводит к крайне низкому минимальному коэффициенту опоры. Остальные элементы алгоритма (перебор ориентаций, использование HeightHandler) аналогичны жадному алгоритму. Вычислительная сложность составляет $O(n \cdot k \cdot m)$.

4.5 Алгоритм FFD (First Fit Decreasing)

Алгоритм FFDSolver реализует классическую стратегию First Fit Decreasing [7], при которой коробки предварительно сортируются по убыванию объёма $V = l \times w \times h$. После сортировки применяется тот же подход размещения, что и в GreedySolver: на каждом шаге выбирается позиция с минимальной итоговой высотой.

Предварительная сортировка по объёму обеспечивает размещение крупных коробок первыми, что позволяет сформировать более плотное основание паллеты. Аналогично алгоритму DBLF, FFDSolver не проверяет ограничение на устойчивость. Вычислительная сложность алгоритма составляет $O(n \log n + n \cdot k \cdot m)$, где первое слагаемое обусловлено сортировкой.

4.6 Метаэвристический алгоритм на основе поиска в большой окрестности

Для повышения качества укладки разработан метаэвристический алгоритм LNSSolver, основанный на методе поиска в большой окрестности (Large Neighborhood Search, LNS) [13]. Данный метод итеративно модифицирует текущее решение с помощью операторов мутации и принимает улучшающие изменения.

Решение кодируется классом GenomHandler (порядок коробок по паллетам и порог минимальной опоры). Каждый элемент массива содержит идентификатор коробки, выбранную ориентацию (-1 означает перебор всех) и предпочтительный угол паллеты для размещения. Порядок

элементов в массиве определяет порядок укладки коробок: перестановка элементов приводит к другому варианту укладки.

Функция симуляции принимает закодированное решение и выполняет укладку согласно жадному алгоритму с учётом заданных параметров. Для каждой коробки рассматриваются указанные ориентации, формируется множество потенциальных позиций, выполняется фильтрация позиций по ограничению на устойчивость и по порогу минимальной опоры, и выбирается позиция с минимальной итоговой высотой. При равных высотах выбирается позиция ближе к предпочтительному углу. Результатом симуляции является укладка и набор метрик качества.

Для исследования пространства решений реализовано тринадцать операторов мутации. Операторы изменения порядка включают реверс подмассива, случайную перестановку, обмен двух элементов и обмен соседних элементов. Операторы сортировки упорядочивают коробки по площади основания, объёму, высоте или весу. Операторы изменения параметров модифицируют ориентации и предпочтительные углы размещения. Специальные операторы включают изменение порога минимальной опоры, группировку коробок одного типа и перемещение коробки с наименьшей опорой в начало последовательности.

Псевдокод алгоритма LNSSolver представлен в алгоритме 2. На каждой итерации выбирается случайный оператор мутации, применяется к копии текущего решения, и выполняется симуляция. Если новое решение улучшает целевую функцию, оно принимается как текущее.

Algorithm 2: Large Neighborhood Search algorithm

Input: $testData$ – problem data, $timeLimit$ – time limit

Output: $bestAnswer$ – best found solution

```
1  params ← initialize with default order;
2  bestResult ← simulate(testData, params);
3  while currentTime < timeLimit do
4      newParams ← copy of params;
5      mutation ← select random mutation operator;
6      apply mutation to newParams;
7      newResult ← simulate(testData, newParams);
8      if newResult.score > bestResult.score then
9          bestResult ← newResult;
10         params ← newParams;
11 return bestResult.answer
```

Целевая функция вычисляется как взвешенная сумма метрик:

$$score = w_1 \cdot percolation + w_2 \cdot min_support_ratio + w_3 \cdot (1 - CoM_z^{rel}) + w_4 \cdot height_balan$$

где $w_1, \ldots, w_4$ — настраиваемые веса, CoM_z^{rel} — нормализованная z-координата центра масс, $height_balance$ — баланс высот между паллетами (см. раздел 4.8). Для задач с одной паллетой используются только первые два слагаемых ($w_3 = w_4 = 0$). Алгоритм использует жадный критерий принятия: новое решение принимается только в случае улучшения значения целевой функции.

Важной оптимизацией LNSSolver является выборочный пересчёт: после применения мутации пересчитываются только те паллеты, содержимое которых было изменено. Это достигается с помощью маски «грязных» паллет (pallet_dirty), что существенно ускоряет работу в мультипаллетном режиме.

4.7 Генетический алгоритм

Разработан генетический алгоритм GeneticSolver, использующий ту же модель кодирования решения (GenomHandler) и те же 13 операторов мутации, что и LNSSolver, но применяющий популяционный подход.

Алгоритм поддерживает популяцию из $P = 24$ особей. Начальная популяция формируется случайной перестановкой порядка коробок. На каждой итерации из лучших 8 особей популяции случайным образом выбирается родитель, к которому применяется оператор мутации, после чего выполняется полная симуляция укладки. Полученное потомство добавляется в новое поколение. Лучшая найденная особь всегда сохраняется (элитизм).

В отличие от LNSSolver, генетический алгоритм на каждой итерации пересчитывает все паллеты заново (вызов rebuild_all), поскольку особи популяции независимы. Это приводит к тому, что GeneticSolver выполняет меньше итераций за единицу времени по сравнению с LNSSolver, особенно при большом числе паллет.

4.8 Укладка на несколько паллет

Для обработки ситуаций, когда груз необходимо разместить на нескольких паллетах, алгоритмы обобщены на мультипаллетный режим. Количество доступных паллет задаётся параметром P, а коробки равномерно распределяются между паллетами.

В метаэвристических алгоритмах (LNSSolver и GeneticSolver) операторы мутации могут модифицировать как порядок коробок внутри одной паллеты, так и перемещать коробки между паллетами (оператор swap_boxes_across_pallets).

Для оценки равномерности загрузки паллет введена метрика баланса высот:

$$height_balance = \max\left(0,\, 1 - \frac{\sigma(H_1, \ldots, H_P)}{\mu(H_1, \ldots, H_P)}\right),$$

где H_i — высота штабеля на i-й паллете, σ — стандартное отклонение, μ — среднее значение. Метрика принимает значение 1 при одинаковых

высотах и стремится к 0 при сильном разбросе. Параметр w_4 в целевой функции управляет значимостью баланса высот при оптимизации.

4.9 Метрики качества укладки

Для оценки качества укладки используются две метрики. Коэффициент заполнения (percolation) определяется как отношение суммарного объёма коробок к объёму паллеты: $percolation = V_{boxes}/(L \times W \times H)$, где L, W, H — длина, ширина и высота паллеты соответственно.

Минимальный коэффициент опоры (min_support_ratio) вычисляется как минимальное значение коэффициента опоры среди всех коробок. Коэффициент опоры отдельной коробки определяется как отношение площади основания, опирающейся на нижележащий слой, к полной площади основания коробки.

4.10 Результаты экспериментов

Экспериментальное тестирование проводилось на наборе из 436 реальных задач. Размеры паллеты составляли 1200 × 800 мм. На рисунке 4.2 представлен пример построенной паллеты, демонстрирующий качество укладки, достигаемое разработанным алгоритмом LNSSolver. Укладка характеризуется плотным заполнением пространства и устойчивым размещением коробок.

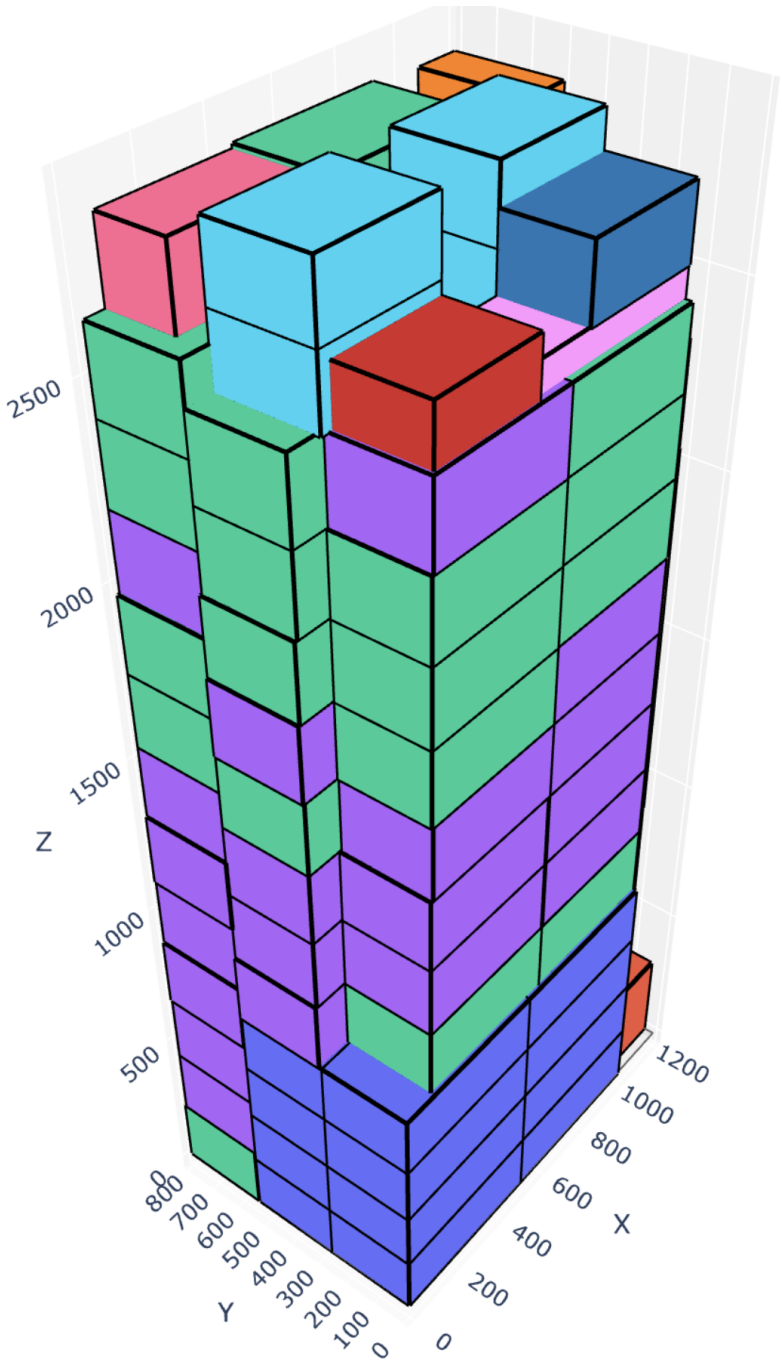

Рисунок 4.2 — Пример построенной паллеты: трёхмерная визуализация результата работы алгоритма LNSSolver

Видеозапись процесса симуляции укладки паллеты, демонстрирующая последовательное размещение коробок сверху вниз, доступна по ссылке: `https://disk.360.yandex.ru/d/xRs8nw0dt8-esw`.

Исходный код разработанного программного решения размещён в открытом репозитории: `https://github.com/Straple/Palletizer`.

На рисунке 4.3 представлена зависимость качества укладки от веса w_2 при фиксированном $w_1 = 1{,}0$ и временном лимите 10 секунд.

Видно, что увеличение веса w_2 приводит к росту минимального коэффициента опоры с 28% до 69%, однако при этом коэффициент заполнения снижается с 71% до 63%. Значение $w_2 = 2{,}0$ обеспечивает баланс между заполнением и устойчивостью.

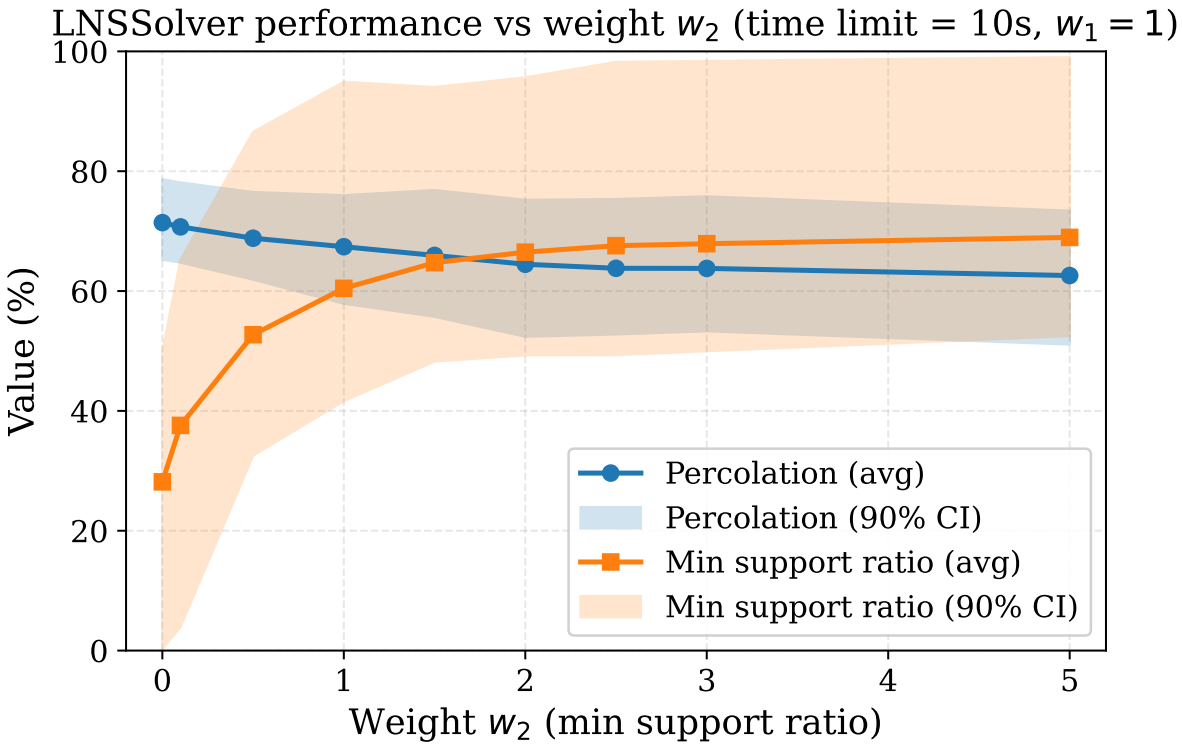

Рисунок 4.3 — Зависимость качества укладки от веса w_2 (время = 10с, $w_1 = 1$)

На рисунке 4.4 представлена зависимость качества укладки от временного лимита при фиксированных весах $w_1 = 1{,}0$, $w_2 = 2{,}0$. С увеличением времени выполнения с 0,5 до 60 секунд коэффициент заполнения возрастает с 56% до 67%, а минимальный коэффициент опоры — с 59% до 69%.

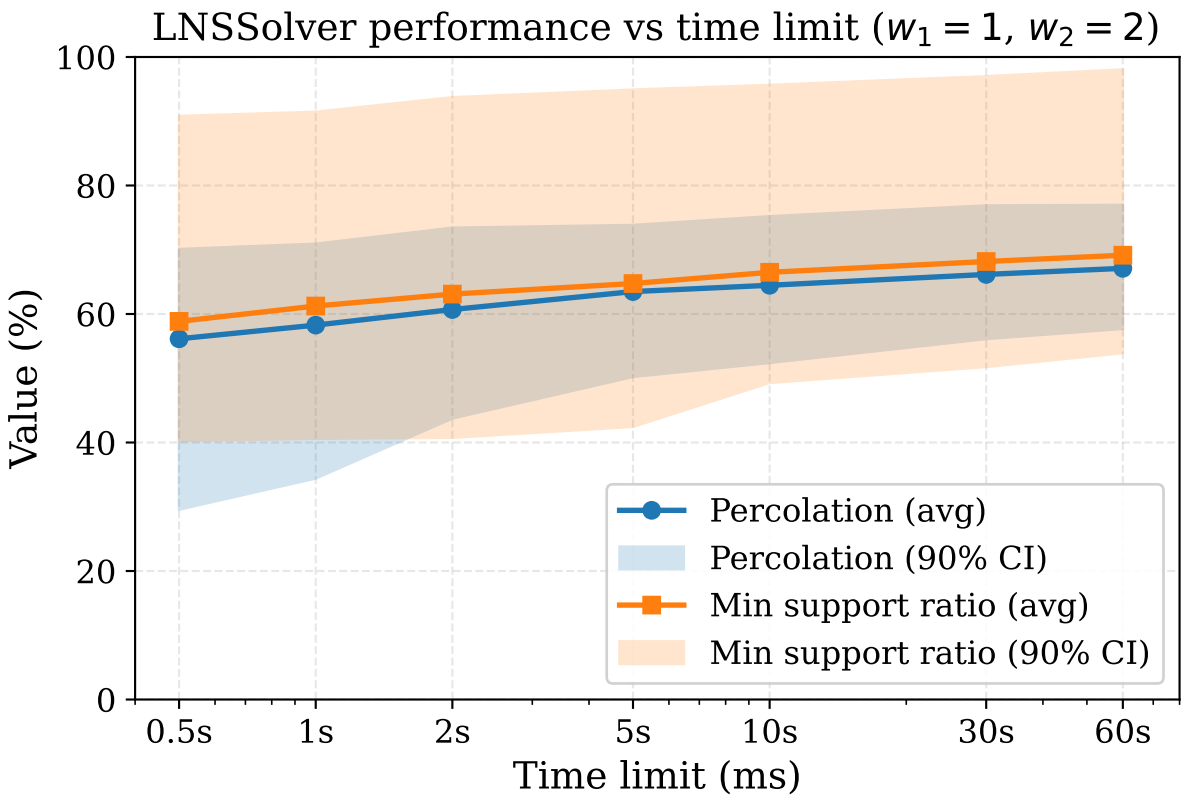

Рисунок 4.4 — Зависимость качества укладки от времени выполнения ($w_1 = 1$, $w_2 = 2$)

Для оценки влияния ограничения на устойчивость было проведено сравнение алгоритма LNSSolver с включённым и отключённым ограничением (таблица 4.3). При отключении ограничения коэффициент заполнения возрастает до 76,69%, однако минимальный коэффициент опоры падает до 2,86%, что неприемлемо для практического применения.

Таблица 4.3 — Влияние ограничения на устойчивость (30 сек на тест, $w_1 = 1$, $w_2 = 0$)

Режим	**Percolation**	**Min support**
С ограничением на устойчивость	69,45%	32,58%
Без ограничения на устойчивость	76,69%	2,86%

В таблице 4.4 представлено сравнение всех пяти разработанных алгоритмов. Тестирование проводилось на 436 реальных задачах с одной паллетой, временной лимит 5 секунд ($w_1 = 1{,}0$, $w_2 = 2{,}0$).

Таблица 4.4 — Сравнение алгоритмов на задачах с одной паллетой (436 тестов, 5 с)

Алгоритм	**Percolation, %**	**Min support, %**	**Время, мс**
GreedySolver	56,04	30,31	12,8
DBLFSolver	65,19	2,44	7,5
FFDSolver	66,36	2,46	8,3
GeneticSolver	63,45	62,02	5130,6
LNSSolver	63,49	64,74	5008,5

На рисунке 4.5 наглядно представлено сравнение алгоритмов с 90%-ными доверительными интервалами.

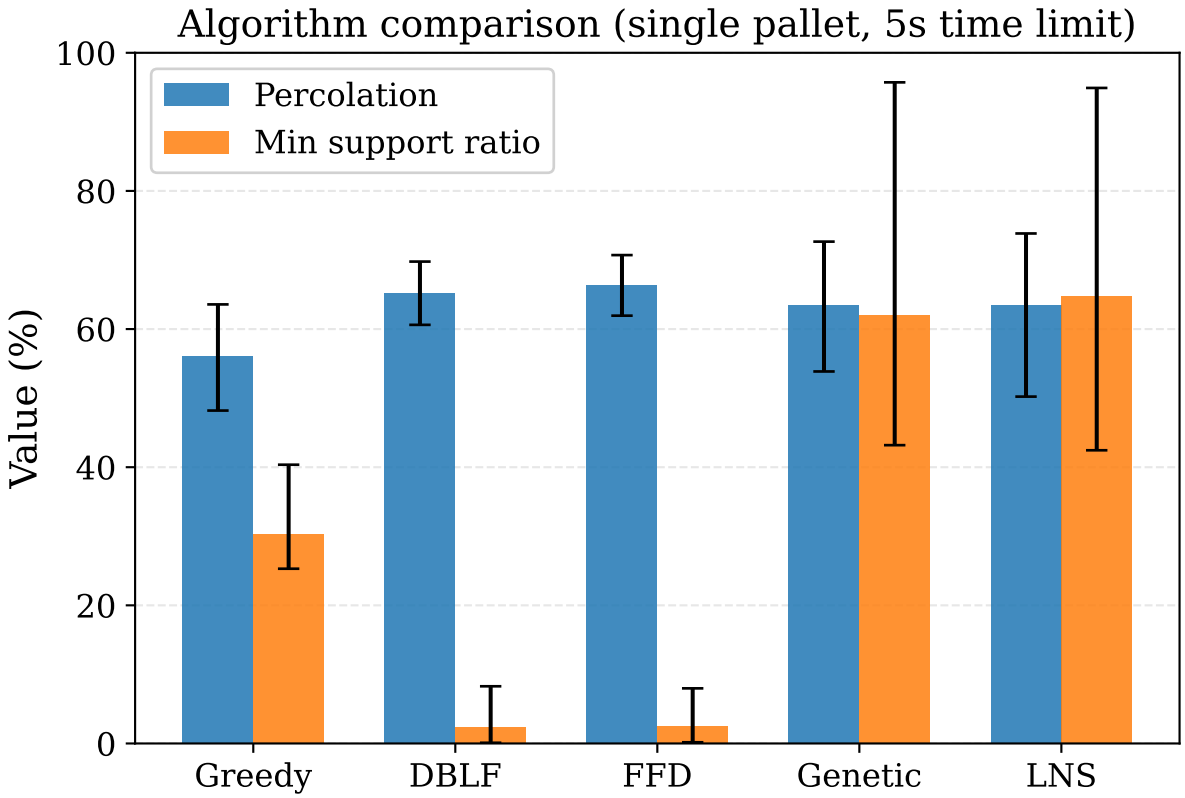

Рисунок 4.5 — Сравнение алгоритмов: перколяция и минимальный коэффициент опоры (одна паллета, 5 с)

Результаты позволяют выделить две группы алгоритмов. Детерминированные эвристики (DBLFSolver, FFDSolver) достигают наилучшего коэффициента заполнения (65–66%) при времени выполнения менее 10 мс, однако минимальный коэффициент опоры составляет лишь 2,4%, что неприемлемо для практического применения. Низкая устойчивость обусловлена отсутствием проверки ограничения на поддержку центра масс.

Метаэвристические алгоритмы (LNSSolver, GeneticSolver) обеспечивают сбалансированные результаты по обеим метрикам. LNSSolver и GeneticSolver показывают близкие значения коэффициента заполнения (≈63,5%), однако LNSSolver превосходит GeneticSolver по минимальному коэффициенту опоры (64,74% против 62,02%). Это объясняется тем, что LNSSolver использует оптимизацию выборочного пересчёта и выполняет больше итераций за единицу времени.

GreedySolver демонстрирует промежуточный коэффициент заполнения (56,04%) с умеренным коэффициентом опоры (30,31%) и является наиболее быстрым алгоритмом, использующим ограничение на устойчивость.

4.11 Сравнение LNSSolver и GeneticSolver при различных временных лимитах

В таблице 4.5 представлено сравнение двух метаэвристических алгоритмов при различных временных лимитах.

Таблица 4.5 — Сравнение LNSSolver и GeneticSolver при различных временных лимитах (одна паллета)

Время, с	**LNSSolver**		**GeneticSolver**	
	Perc, %	Supp, %	Perc, %	Supp, %
0,5	56,14	58,85	60,00	47,41
1	58,27	61,24	60,10	52,96
2	60,70	63,10	61,73	56,90
5	63,49	64,74	63,45	62,02
10	64,47	66,49	65,19	63,84
30	66,16	68,17	66,93	66,48
60	67,11	69,15	67,38	68,48

На рисунке 4.6 представлена визуализация динамики сходимости обоих алгоритмов.

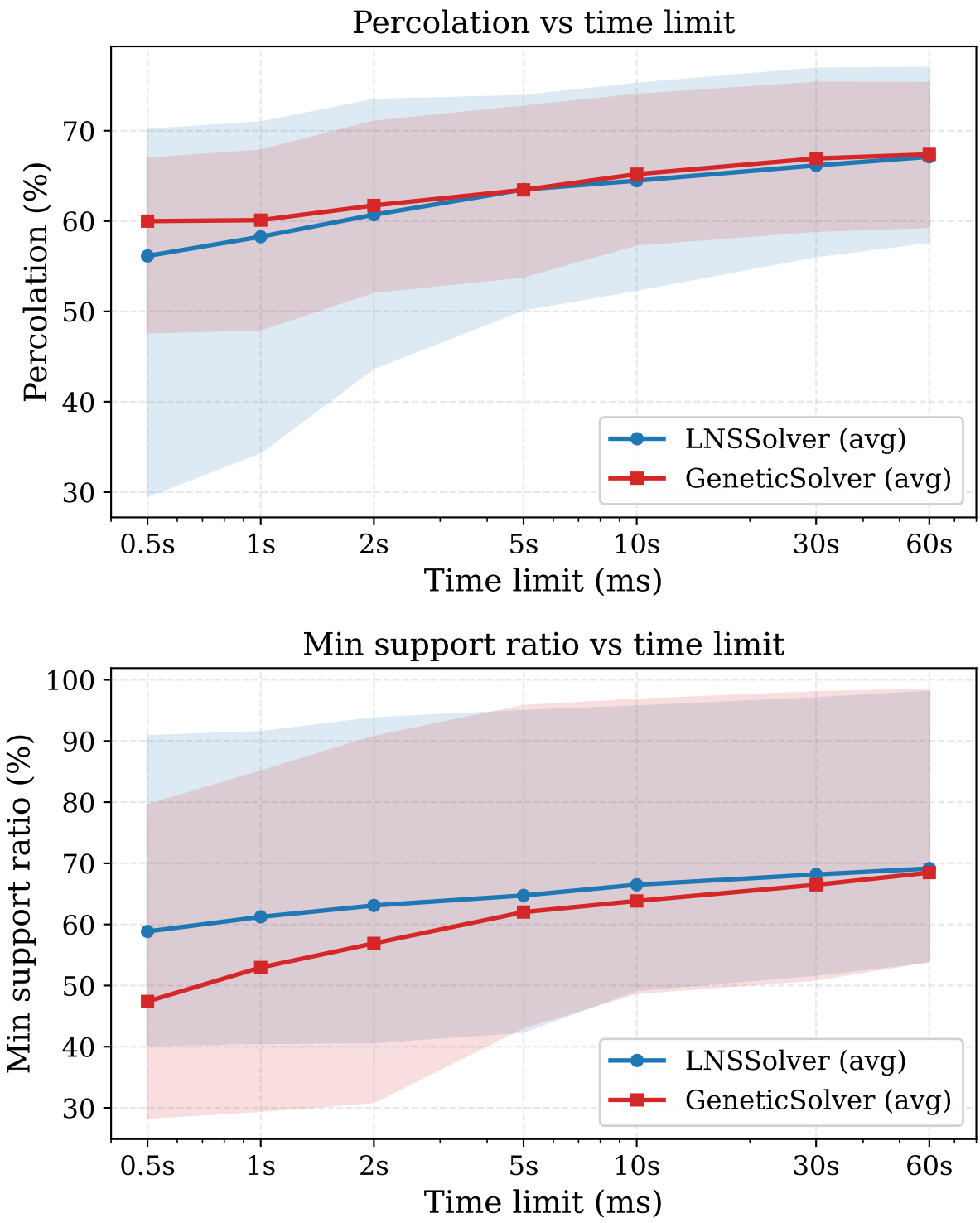

Рисунок 4.6 — Сравнение LNSSolver и GeneticSolver: перколяция и минимальный коэффициент опоры при различных временных лимитах

При коротких временных лимитах (0,5–2 с) LNSSolver существенно превосходит GeneticSolver по минимальному коэффициенту опоры, что делает его предпочтительным для практических задач с жёсткими ограничениями на время. При увеличении лимита до 60 с разрыв сокращается, и GeneticSolver достигает сопоставимых результатов. Попу-

ляционный подход GeneticSolver демонстрирует лучший коэффициент заполнения на коротких интервалах за счёт разнообразия начальной популяции.

4.12 Результаты в мультипаллетном режиме

В таблице 4.6 представлены результаты алгоритмов в мультипаллетном режиме (временной лимит 5 с, $w_1 = 1{,}0$, $w_2 = 2{,}0$).

Таблица 4.6 — Сравнение алгоритмов в мультипаллетном режиме (5 с)

Алгоритм	**Perc, %**	**Supp, %**	**Height bal, %**	**Время, мс**
GreedySolver	55,33	19,08	83,57	38,2
DBLFSolver	64,60	0,44	85,36	21,2
FFDSolver	65,22	0,42	52,16	20,9
GeneticSolver	59,01	44,06	84,83	5470,6
LNSSolver	62,09	42,30	84,26	5016,7

В мультипаллетном режиме FFDSolver демонстрирует критически низкий баланс высот (52,16%), поскольку при сортировке по объёму крупные коробки концентрируются на первых паллетах. Алгоритмы с метаэвристической оптимизацией обеспечивают баланс высот на уровне 84–85%. LNSSolver сохраняет лидерство по коэффициенту заполнения среди метаэвристик (62,09% против 59,01% у GeneticSolver) благодаря оптимизации выборочного пересчёта паллет.

4.13 Влияние веса баланса высот в мультипаллетном режиме

На рисунке 4.7 представлена зависимость метрик качества от веса w_4 (баланс высот) при фиксированных $w_1 = 1{,}0$, $w_2 = 2{,}0$ и временном лимите 10 с.

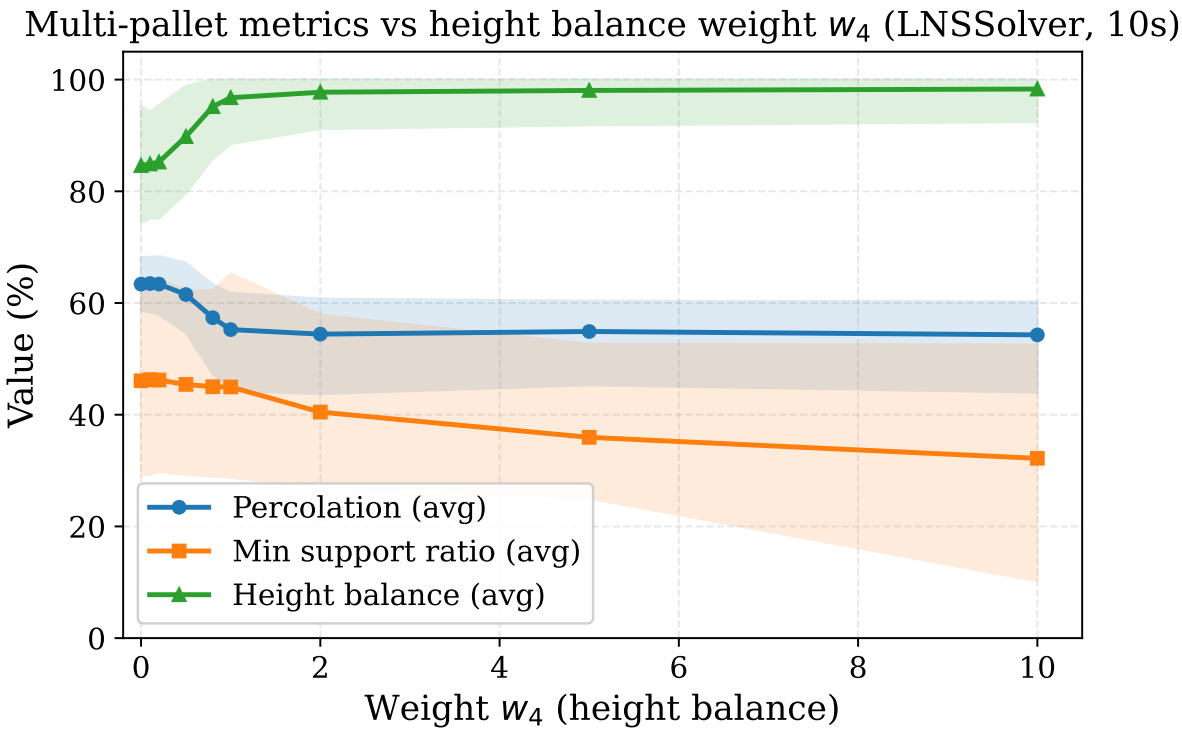

Рисунок 4.7 — Зависимость метрик качества от веса w_4 в мультипаллетном режиме (LNSSolver, 10 с)

Увеличение w_4 приводит к росту баланса высот с 85% до 98%, однако за счёт снижения перколяции (с 63% до 54%) и минимального коэффициента опоры (с 46% до 32%). Значения $w_4 \in [0{,}2;\ 1{,}0]$ обеспечивают оптимальный компромисс: баланс высот превышает 85% при умеренном снижении остальных метрик.

5 Анализ результатов

В данной главе представлен сводный анализ экспериментальных результатов разработанных алгоритмов укладки.

5.1 Сравнение алгоритмов для одной паллеты

Тестирование проводилось на наборе из 436 реальных задач. Размеры паллеты: 1200 × 800 мм, нормировочная высота: 2200 мм. Временной лимит 5 с для метаэвристических алгоритмов, веса целевой функции $w_1 = 1{,}0$, $w_2 = 2{,}0$.

Таблица 5.1 — Сравнение алгоритмов на тестовом датасете (одна паллета, 436 тестов)

Алгоритм	**Перколяция, %**	**Мин. опора, %**	**Устойчивость**	**Время, м**
GreedySolver	56,04	30,31	да	12,8
DBLFSolver	65,19	2,44	нет	7,5
FFDSolver	66,36	2,46	нет	8,3
GeneticSolver	63,45	62,02	да	5130,6
LNSSolver	63,49	64,74	да	5008,5

Алгоритмы разделяются на две группы по критерию наличия ограничения на устойчивость. Без ограничения (DBLFSolver, FFDSolver) достигается наивысшая перколяция (65–66%), но минимальный коэффициент опоры падает ниже 3%. Метаэвристические алгоритмы (LNSSolver, GeneticSolver) обеспечивают баланс: перколяция ≈63,5% при минимальном коэффициенте опоры свыше 62%.

LNSSolver является лучшим алгоритмом по совокупности метрик при ограничении на время порядка 5 с: он превосходит GreedySolver на 7,45 п.п. по перколяции и на 34,43 п.п. по минимальному коэффициенту опоры, а также немного опережает GeneticSolver по обоим показателям.

5.2 Результаты в мультипаллетном режиме

Мультипаллетный режим тестировался на том же наборе тестов, объединённых в группы по нескольку заказов.

Таблица 5.2 — Сравнение алгоритмов в мультипаллетном режиме (5 с)

Алгоритм	**Перколяция, %**	**Мин. опора, %**	**Баланс высот, %**	**Врем**
GreedySolver	55,33	19,08	83,57	38
DBLFSolver	64,60	0,44	85,36	21
FFDSolver	65,22	0,42	52,16	20
GeneticSolver	59,01	44,06	84,83	547
LNSSolver	62,09	42,30	84,26	501

В мультипаллетном режиме FFDSolver показывает критически низкий баланс высот (52%), поскольку крупные коробки концентрируются на первых паллетах. LNSSolver демонстрирует лучшую перколяцию среди метаэвристик (62,09%), сохраняя высокий баланс высот (84,26%) и приемлемый коэффициент опоры.

Заключение

По результатам работы решены следующие задачи:

— выполнена формальная постановка задачи трёхмерной паллетизации и введены метрические показатели: коэффициент заполнения (percolation), минимальный коэффициент опоры (min_support_ratio) и баланс высот (height_balance);

— разработана и программно реализована вспомогательная структура данных HeightHandler для хранения карты высот с пятью различными реализациями (Baseline, Rects, Grid, SegTree, Quadtree);

— разработано пять алгоритмов укладки:

GreedySolver — жадный алгоритм с ограничением на устойчивость (56,04% заполнения, 30,31% опоры, 12,8 мс);

DBLFSolver — алгоритм Deepest Bottom Left Fill (65,19% заполнения, 7,5 мс);

FFDSolver — алгоритм First Fit Decreasing (66,36% заполнения, 8,3 мс);

LNSSolver — метаэвристический алгоритм на основе LNS с тринадцатью операторами мутации (63,49% заполнения, 64,74% опоры);

GeneticSolver — генетический алгоритм с популяционным подходом (63,45% заполнения, 62,02% опоры);

— реализована поддержка мультипаллетного режима с метрикой баланса высот;

— проведено экспериментальное тестирование на 436 реальных задачах.

Экспериментальное сравнение пяти реализаций структуры HeightHandler показало, что простая реализация Rects на основе отсортированного списка прямоугольников превосходит более сложные структуры данных по производительности в 22 раза по сравнению с базовой реализацией.

Сравнительный анализ разработанных алгоритмов выявил две характерные группы. Детерминированные эвристики (DBLFSolver,

FFDSolver) достигают наивысшего коэффициента заполнения (65–66%) при времени выполнения менее 10 мс, однако не обеспечивают устойчивость размещения (минимальный коэффициент опоры $\approx$ 2,4%). Метаэвристические алгоритмы (LNSSolver, GeneticSolver) обеспечивают баланс между заполнением и устойчивостью. LNSSolver при лимите 5 с достигает коэффициента заполнения 63,49% и минимального коэффициента опоры 64,74%, превосходя GreedySolver на 7,45 и 34,43 процентных пункта соответственно. LNSSolver превосходит GeneticSolver на коротких временных интервалах благодаря оптимизации выборочного пересчёта паллет.

В мультипаллетном режиме LNSSolver демонстрирует перколяцию 62,09% при балансе высот 84,26%. FFDSolver показывает критически низкий баланс высот (52,16%) из-за концентрации крупных коробок на первых паллетах.

Разработанное программное решение может быть интегрировано в WMS-систему склада для автоматической генерации схем паллетизации. Время выполнения алгоритма LNSSolver порядка 5 секунд на паллету является приемлемым для задач складской логистики.

В дальнейшем планируется исследовать адаптивный выбор весов операторов мутации (Adaptive LNS), обучение эвристики при помощи методов RL для online-сценариев, а также гибридный подход, комбинирующий высокую перколяцию FFDSolver с устойчивостью метаэвристических методов.

Список использованных источников

1. *Martello, Silvano.* The Three-Dimensional Bin Packing Problem / Silvano Martello, David Pisinger, Daniele Vigo // *Operations Research.* — 2000. — Vol. 48, no. 2. — Pp. 256–267.

2. *Wascher, Gerhard.* An Improved Typology of Cutting and Packing Problems / Gerhard Wascher, Heike Haußner, Holger Schumann // *European Journal of Operational Research.* — 2007. — Vol. 183, no. 3. — Pp. 1109–1130.

3. *Bortfeldt, Andreas.* Constraints in Container Loading – A State-of-the-Art Review / Andreas Bortfeldt, Gerhard Wascher // *European Journal of Operational Research.* — 2013. — Vol. 229, no. 1. — Pp. 1–20.

4. *Karabulut, K.* A hybrid genetic algorithm for packing in 3D with deepest bottom left with fill method / K. Karabulut, M. M. İnceoglu // International Conference on Advances in Information Systems. — Springer, 2004. — Pp. 441–450.

5. *Wei, Lijun.* A Skyline-Based Heuristic for the 2D Rectangular Strip Packing Problem / Lijun Wei, Andrew Lim, Wenbin Zhu // IEA/AIE 2011. — 2011.

6. A Maximal-Space Algorithm for the Container Loading Problem / Francisco Parreno, Ramón Alvarez-Valdes, José M. Tamarit, José F. Oliveira // *INFORMS Journal on Computing.* — 2008. — Vol. 20, no. 3. — Pp. 412–422.

7. *Coffman Jr., Edward G.* Approximation Algorithms for Bin Packing: A Survey / Edward G. Coffman, Jr., Michael R. Garey, David S. Johnson // Approximation Algorithms for NP-hard Problems / Ed. by Dorit S. Hochbaum. — PWS Publishing Co., 1996. — Pp. 46–93.

8. *Jylanki, Jukka.* A Thousand Ways to Pack the Bin — A Practical Approach to Two-Dimensional Rectangle Bin Packing: Technical Report T-106 / Jukka Jylanki: Aalto University, School of Science and Technology, 2010.

9. *О. П. Тимофеева Т. Ю. Чернышева, А. В. Волков.* Генетический алгоритм в оптимизации трёхмерной упаковки блоков

в контейнер / А. В. Волков О. П. Тимофеева, Т. Ю. Чернышева, О. Н. Корелин // Нижегородский государственный технический университет им. Р. Е. Алексеева. — 2017.

10. A GAN-Based Genetic Algorithm for Solving the 3D Bin Packing Problem / Boliang Zhang, Yu Yao, H. K. Kan, Wuman Luo // *Scientific Reports*. — 2024. — Vol. 14. — P. 7775.

11. *Гиля Зетинов, А.А.* Разработка алгоритма укладки паллет на полностью автоматизированном складе / А.А. Гиля Зетинов, А.В. Хельвас, К.К. Панкратов // *Труды Московского физико-технического института*. — 2019. — Т. 11, № 1(41). — С. 20–30.

12. *Gilya-Zetinov, A.* Palletizing for Full-automated Warehouses on the Genetics Algorithm Base / A. Gilya-Zetinov, D. Demianova, A. Khelvas // International Conference on Engineering and Telecommunication (EnT) Proceedings / Ed. by 2019 International Conference on Engineering, Telecommunication (EnT). — IEEE, 2019. — Pp. 1–5.

13. *Pisinger, David*. A general heuristic for vehicle routing problems / David Pisinger, Stefan Ropke // *Computers & Operations Research*. — 2005. — 01. — Vol. 34. — Pp. 2403–2435.

14. *Ropke, Stefan*. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows / Stefan Ropke, David Pisinger // *Transportation Science*. — 2006. — 11. — Vol. 40. — Pp. 455–472.

15. *Shaw, Paul*. Using Constraint Programming and Local Search / Paul Shaw. — 2002. — 04.

16. *Pisinger, David*. Large Neighborhood Search / David Pisinger, Stefan Ropke. — 2010. — 09. — Pp. 399–419.

17. *Ekici, Ali*. A large neighborhood search algorithm and lower bounds for the variable-Sized bin packing problem with conflicts / Ali Ekici // *European Journal of Operational Research*. — 2023. — Vol. 308, no. 3. — Pp. 1007–1020. `https://www.sciencedirect.com/science/article/pii/S0377221722010256`.

18. *Faroe, Oluf*. Guided Local Search for the Three-Dimensional Bin-Packing Problem / Oluf Faroe, David Pisinger, Martin Zachariasen // *IN-*

FORMS Journal on Computing. — 2003. — Vol. 15, no. 3. — Pp. 267–283.

19. Mathematical Models and Heuristic Algorithms for Pallet Building Problems with Practical Constraints / Gabriele Calzavara, Manuel Iori, Marco Locatelli, Mayron Moreira // *Annals of Operations Research.* — 2021.

20. Algorithms for 3D Guillotine Cutting Problems: Unbounded Knapsack, Cutting Stock and Strip Packing / Thiago A. de Queiroz, Flávio K. Miyazawa, Yoshiko Wakabayashi, Eduardo C. Xavier // *Computers & Operations Research.* — 2012. — Vol. 39. — Pp. 728–738.

21. *Amossen, Rasmus R.* Multi-Dimensional Bin Packing Problems with Guillotine Constraints / Rasmus R. Amossen, David Pisinger // *Computers & Operations Research.* — 2010. — Vol. 37, no. 11. — Pp. 1999–2006.

22. *Bischoff, Eberhard E.* Issues in the Development of Approaches to Container Loading / Eberhard E. Bischoff, M. S. W. Ratcliff // *Omega.* — 1995. — Vol. 23, no. 4. — Pp. 377–390.

23. *Fang, Jie.* A Deep Reinforcement Learning Algorithm for the Rectangular Strip Packing Problem / Jie Fang, Yunqing Rao, Mingliang Shi // *PLoS ONE.* — 2023. — Vol. 18, no. 3. — P. e0282598.

24. Learning Practically Feasible Policies for Online 3D Bin Packing / Hang Zhao, Chenyang Zhu, Xin Xu et al. // *arXiv preprint arXiv:2108.13680.* — 2021.

25. On-Line Three-Dimensional Packing Problems: A Review of Off-Line and On-Line Solution Approaches / Sara Ali, Antonio G. Ramos, Maria A. Carravilla, José F. Oliveira // *Computers & Industrial Engineering.* — 2022. — Vol. 168. — P. 108122.

26. *Ananno, Anan A.* A Multi-Heuristic Algorithm for Multi-Container 3-D Bin Packing Problem Optimization Using Real-World Constraints / Anan A. Ananno, Luis Ribeiro // *IEEE Access.* — 2024. — Vol. 12. — Pp. 42105–42125.

27. Algorithms for Pallet Building and Truck Loading in an Interdepot Transportation Problem / Maria T. Alonso, Ramón Alvarez-Valdes, Francisco Parreno, José M. Tamarit // *Mathematical Problems in Engi-*

neering. — 2016. — Vol. 2016. — Pp. 1–19.

28. Optimizing E-Commerce Warehousing through Open Dimension Management in a Three-Dimensional Bin Packing System / Jianglong Yang, Kaibo Liang, Huwei Liu et al. // *PeerJ Computer Science.* — 2023. — Vol. 9. — P. e1613.

29. *McIlroy, Peter.* Optimistic sorting and information theoretic complexity / Peter McIlroy // Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms. — SODA '93. — USA: Society for Industrial and Applied Mathematics, 1993. — Pp. 467–474.

30. *Baumann, Michael.* Method and device for automated commissioning of bundles. — 2013. — 04. — US Patent 8423178B2.

31. *Pankratov, Kirill.* Pallet building system. — 2020. — 07. — US Patent 11613434 B2.

32. International Conference on Engineering and Telecommunication (EnT) Proceedings / Ed. by 2019 International Conference on Engineering, Telecommunication (EnT). — IEEE, 2019.

33. *Lodi, Andrea.* Heuristic Algorithms for the Three-Dimensional Bin Packing Problem / Andrea Lodi, Silvano Martello, Daniele Vigo // *European Journal of Operational Research.* — 2002. — Vol. 141, no. 2. — Pp. 410–420.

34. A Comparative Review of 3D Container Loading Algorithms / Xiaozhou Zhao, Julia A. Bennell, Tolga Bektaş, Kath Dowsland // *International Transactions in Operational Research.* — 2014. — Vol. 21, no. 5. — Pp. 775–804.

35. *Crainic, Teodor Gabriel.* Extreme Point-Based Heuristics for Three-Dimensional Bin Packing: Tech. Rep. CIRRELT-2007-41 / Teodor Gabriel Crainic, Guido Perboli, Roberto Tadei: CIRRELT, 2007.